

El Pasado: Vibe Coding

Suposiciones
de la IA

Deuda
Técnica

Código
Descartable

El Futuro: Spec-Driven Development (SDD)



El Fin del Vibe Coding y el Ascenso del Plano Vivo

Los agentes de IA han cambiado la programación, pero escribir prompts vagos es insostenible a escala. Bienvenidos a la era donde la intención estructurada dicta el código.

Intención
Estructurada

Artefactos
Vivos

Desarrollo Predecible

> prompt: agrega compartir fotos

Tratar a los agentes como motores de búsqueda genera deuda técnica instantánea. La falta de contexto explícito obliga a la IA a adivinar.

! Stack Incorrecto

Asumió MongoDB en lugar de Postgres.
Incompatibilidad de datos.

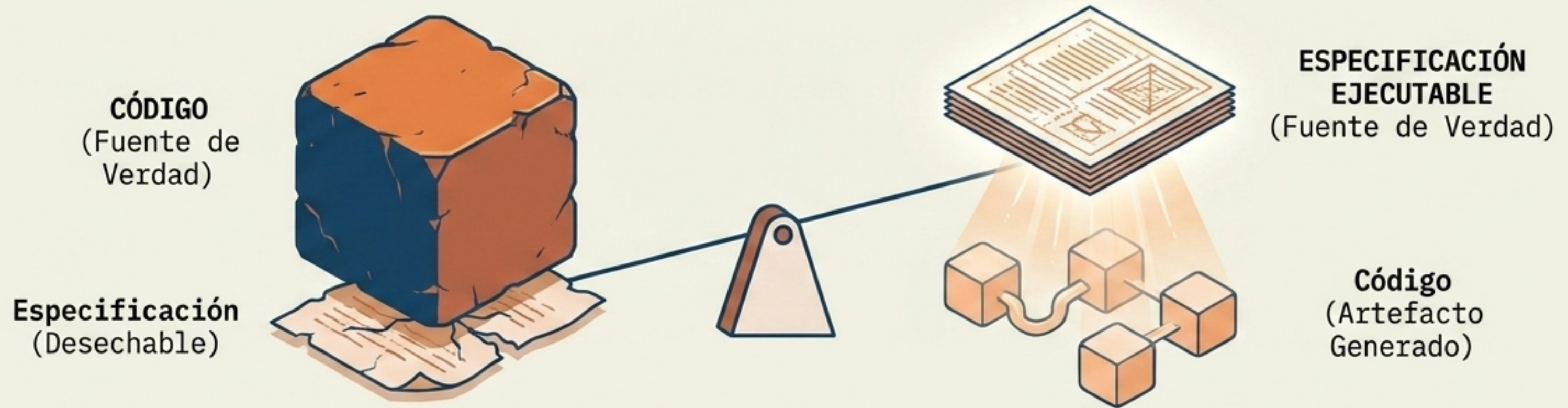
! Sin Seguridad

Expuso la API sin autenticación.
Vulnerabilidad crítica.

! Deuda Cognitiva

Código monolítico de 1000 líneas imposible de mantener.

> La Inversión de Poder: La Intención es la Nueva Fuente de Verdad



Desarrollo Tradicional

El código era el rey. La documentación servía al código y rápidamente quedaba obsoleta.

Spec-Driven Development

La especificación es el artefacto principal. El código es simplemente su última expresión. Refactorizar significa reestructurar la intención, no reescribir la sintaxis.

> Anatomía del Spec-Driven Development (SDD)

Pilar 1: Documentos Vivos

✗ **No es:** Metodología en cascada ni requisitos secos e inamovibles.

✓ **Es:** Documentos iterables que evolucionan junto con tu comprensión del problema.

Pilar 2: Decisiones Explícitas

✗ **No es:** Burocracia que ralentiza el desarrollo.

✓ **Es:** Control de versiones para tu pensamiento. Saca la arquitectura de Slack y la hace ejecutable.

Pilar 3: Motor de Generación

✗ **No es:** Un documento escrito solo para ser leído por humanos.

✓ **Es:** Un contrato estructurado que los agentes de IA utilizan para generar, probar y validar código.

La Evolución del Paradigma de Desarrollo



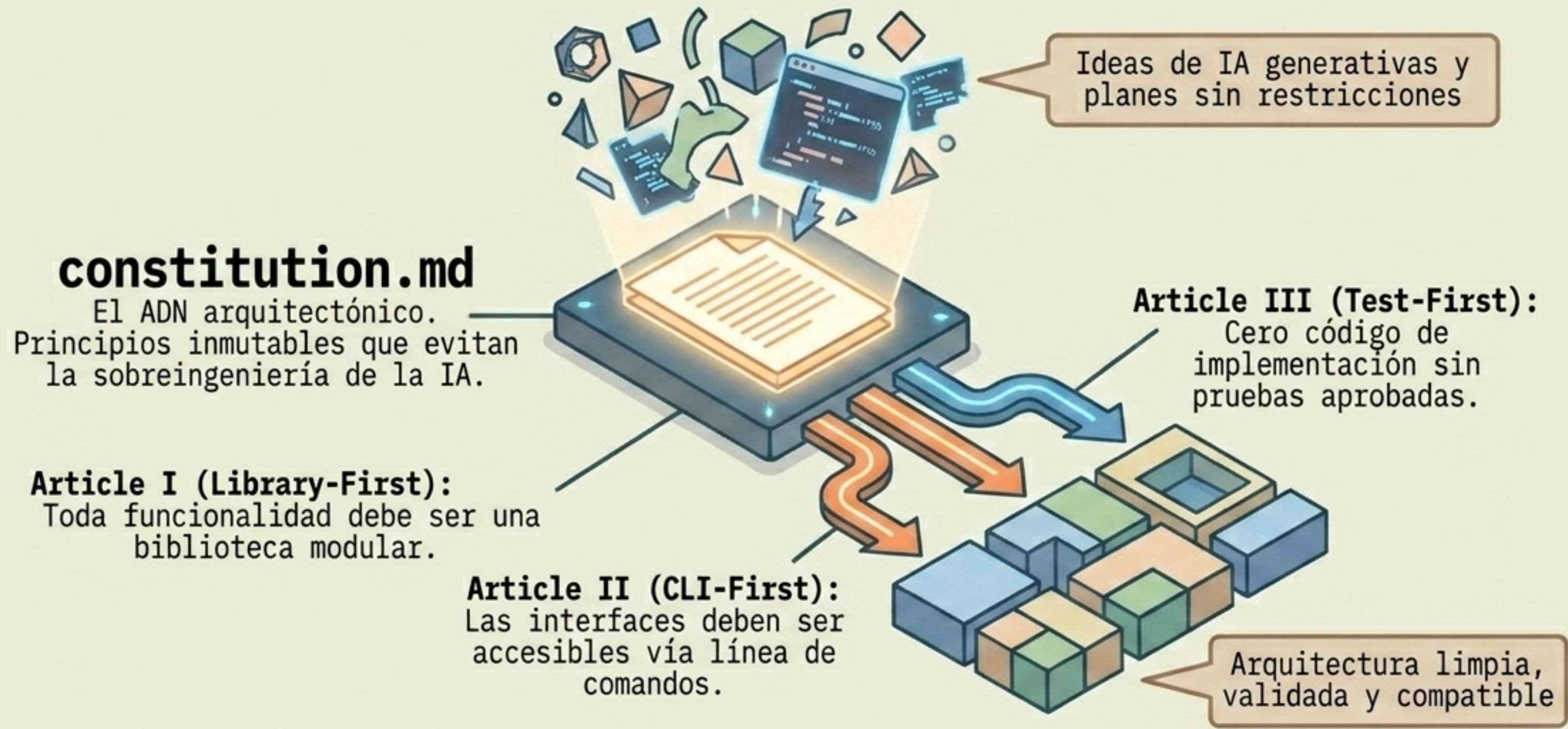
Dimensión	Desarrollo Tradicional	Vibe Coding (IA Inicial)	Spec-Driven Development
Fuente de Verdad	El Código	El Prompt Efímero	El Markdown / Spec
Manejo de Cambios	Refactorización manual lenta	Reescrituras impredecibles	Regeneración sistemática
Medio de Negociación	Reuniones y PRDs aislados	Ensayo y error constante	El documento de especificación iterativo
Rol del Humano	Escritor de sintaxis	Revisor de bloques masivos	Arquitecto de sistemas y validador

El Motor SDD: Un Flujo de Trabajo de 4 Fases

Regla de Oro: Ninguna fase avanza hasta que el artefacto actual haya sido validado por el desarrollador.



El Filtro Constitucional: Protegiendo la Arquitectura



Fase 1: Especificar (Capturando la Intención Pura)

Uso de restricciones estructuradas para evitar que la IA alucine detalles técnicos.

Usuario:

Necesitamos un sistema para que los clientes compartan fotos de sus proyectos terminados. Debe ser rápido y fácil de usar desde el móvil. No te preocupes por el stack técnico aún, solo define el viaje del usuario.

Especificación de Funcionalidad

Historias de Usuario

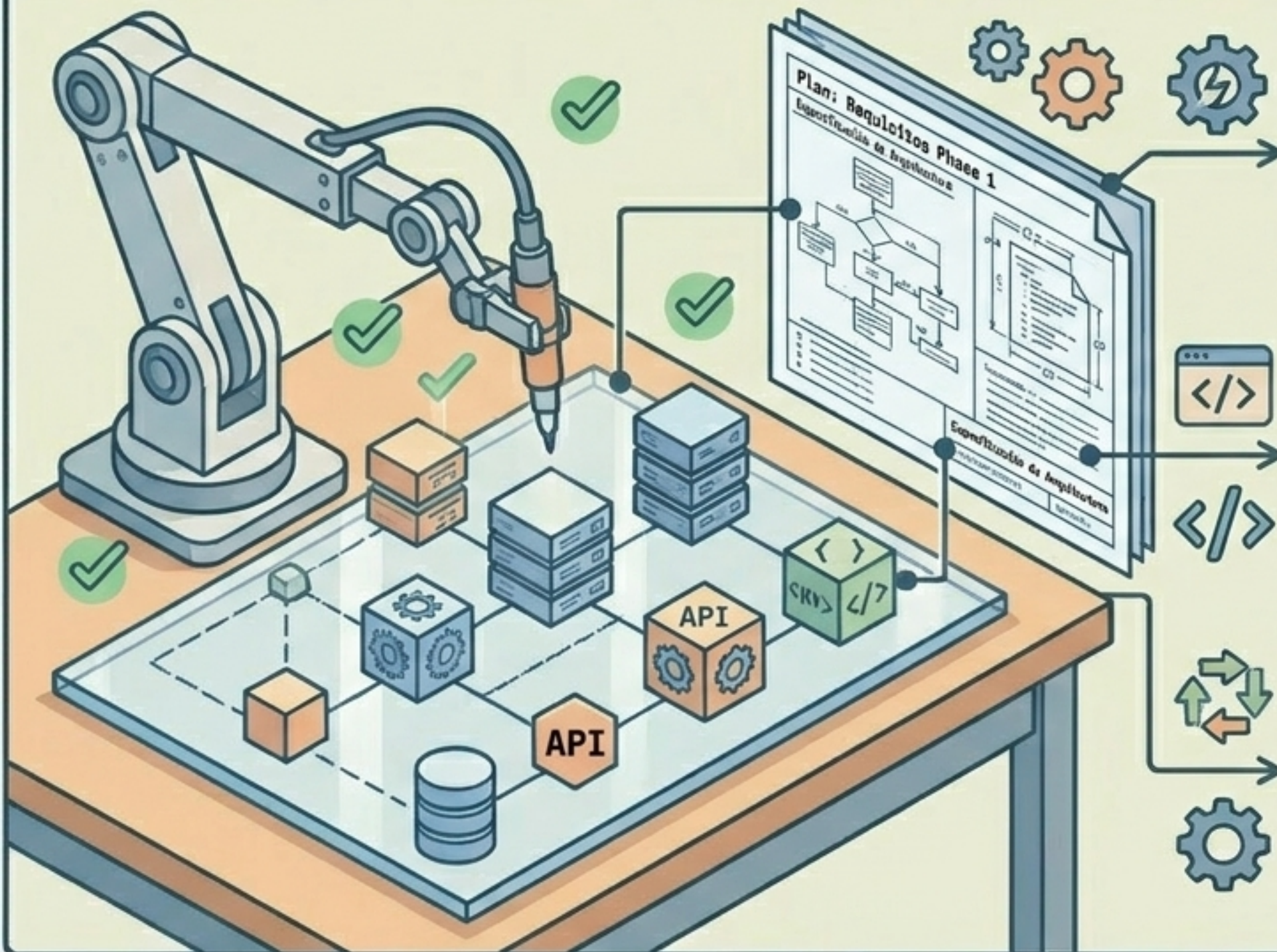
- Como cliente, quiero subir fotos desde mi galería para documentar mi proyecto.

Criterios de Aceptación

1. Subida múltiple soportada.
2. Compresión automática de imágenes.

[NEEDS CLARIFICATION: ¿Método de autenticación requerido para subir fotos?]

Fase 2: Planificar (Traducción Técnica y Restricciones)



Traducción Técnica

El agente toma los requisitos comerciales y genera modelos de datos, contratos de API y variaciones arquitectónicas.

Decisiones Rastreables

Cada elección tecnológica en el plan generado debe vincularse directamente a un requisito de la Fase 1.

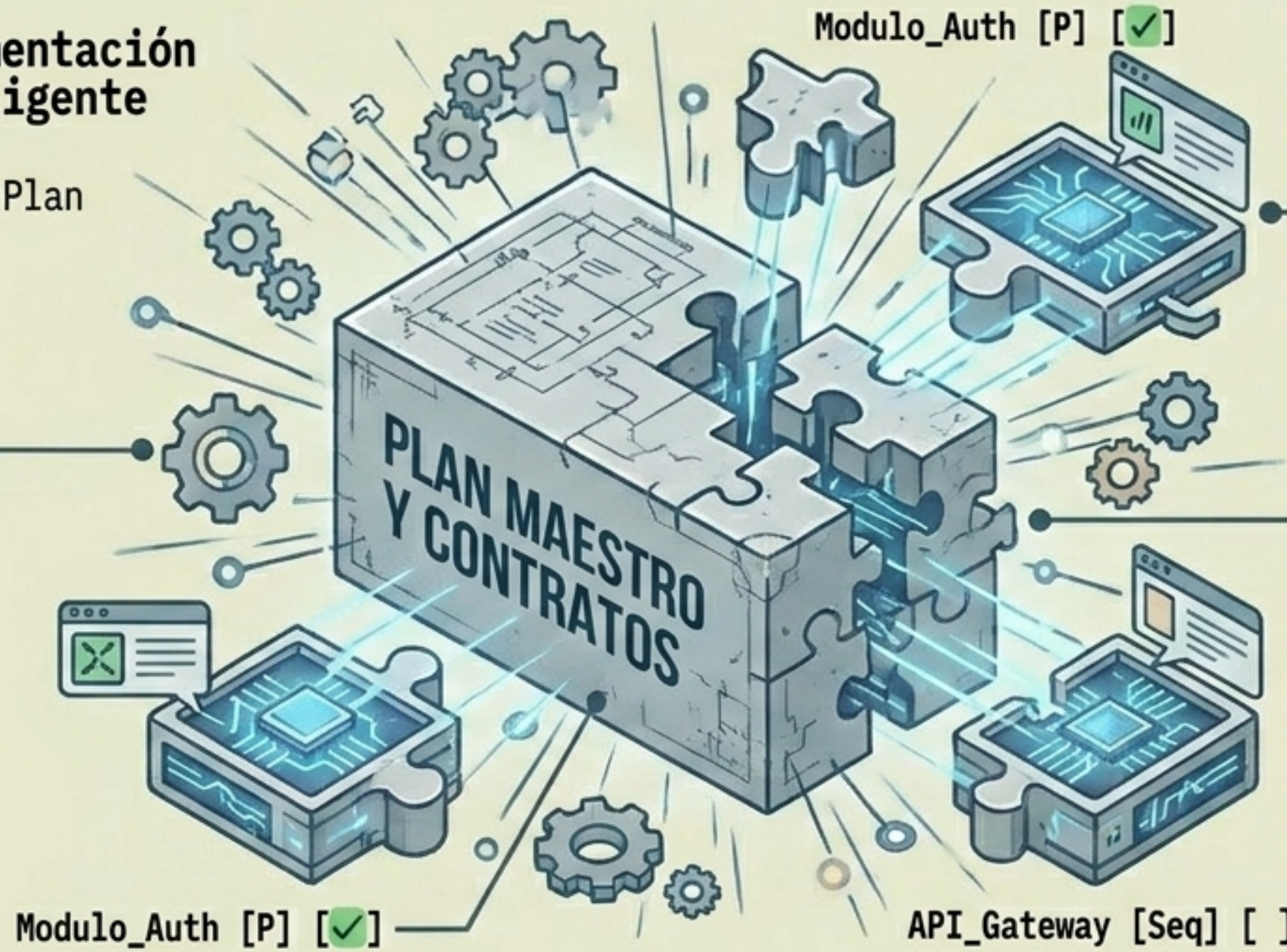
Exploración de Múltiples Vías

La separación de Spec y Plan permite pedir al agente explorar implementaciones paralelas (ej. Rust vs. Go) bajo la misma especificación exacta.

Fases 3 & 4: Tareas e Implementación Enfocada

Fragmentación Inteligente

El agente desglosa el Plan en tareas atómicas específicas.



Identificación de Paralelismo

Etiquetado de tareas independientes ([P]) que múltiples agentes pueden ejecutar simultáneamente.

Revisión Cognitiva

En lugar de revisar volcados de 1000 líneas (deuda cognitiva), el humano revisa implementaciones pequeñas, enfocadas y pre-validadas por pruebas.

El Espectro de Madurez SDD

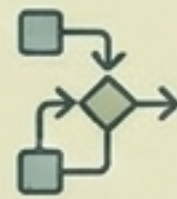
Cómo evoluciona el uso de las especificaciones a lo largo del tiempo.



Matriz de Evaluación de Herramientas (Estado Actual)



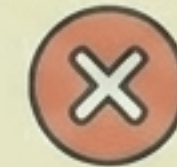
Herramienta



Nivel y Flujo



**Fortalezas
(Pros)**



**Desafíos
(Contras)**

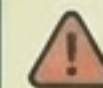


Kiro

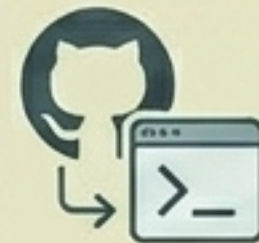
Spec-First.
Req -> Diseño ->
Tareas.



Ligero, intuitivo y
fácil de iniciar.



Puede ser excesivo
para correcciones de
errores pequeños.



**Spec-kit
(GitHub)**

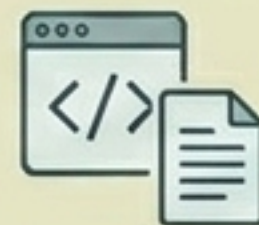
Spec-Anchored.
Comandos (/specify,
/plan) +
Constitución.



Potente, integrado
en IDE, artefactos
altamente
detallados.



Genera un volumen
masivo de archivos
Markdown para
revisar.

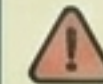


Tessl

Spec-as-Source.
Mapeo 1:1 entre texto
y código.



Visión pura del
futuro
arquitectónico.



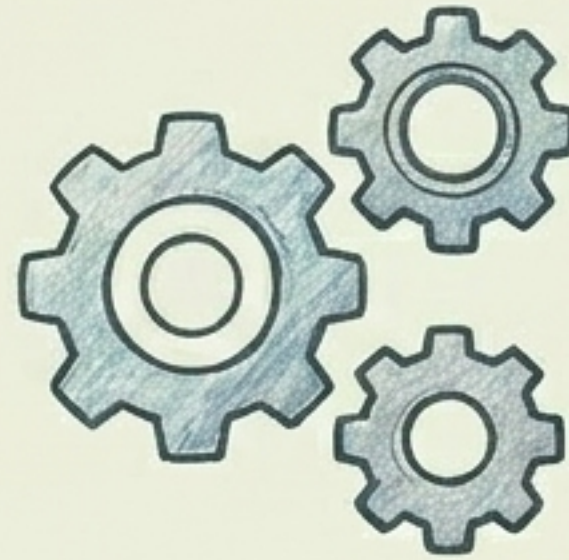
En fase beta, enfrenta
los desafíos
históricos del
Model-Driven
Development (MDD).

¿Dónde Brilla el SDD en el Mundo Real?



Proyectos Greenfield (Cero a Uno)

Evita que la IA construya soluciones genéricas basadas en patrones de código abierto, obligando a una alineación precisa con la intención única del producto desde el día uno.



Sistemas Existentes (N a N+1)

El caso de uso más poderoso. El plan codifica las restricciones arquitectónicas existentes, asegurando que el nuevo código se sienta nativo y no como un parche pegado con cinta.



Modernización de Legacy

Permite aplicar ingeniería inversa al código antiguo para extraer especificaciones limpias, y luego regenerar el sistema desde cero sin heredar la deuda técnica original.

Baño de Realidad: Desafíos del SDD

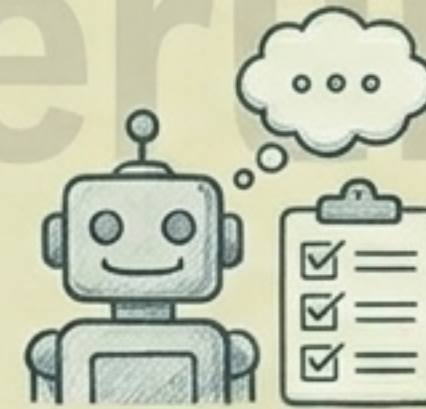
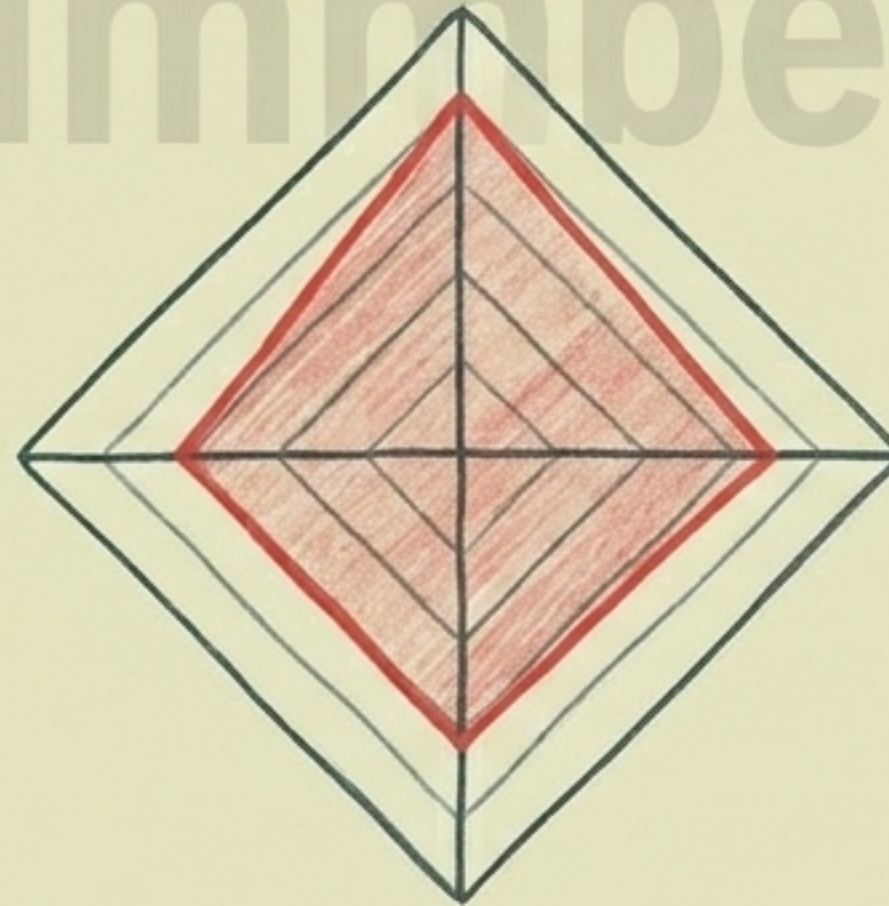


La especificación impulsada por IA no es magia y viene con nuevas fricciones.



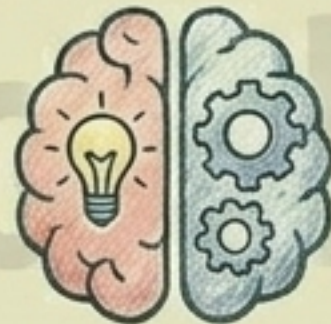
Fatiga de Markdown

Leer, revisar y gestionar docenas de archivos de texto generados puede ser más tedioso y cognitivamente pesado que revisar código real.



Falsa Sensación de Control

Las ventanas de contexto inmensas no garantizan obediencia. Los agentes aún ignoran instrucciones o se entusiasman demasiado cumpliendo reglas constitucionales al pie de la letra.



El Dilema Funcional vs Técnico

A los equipos históricamente les cuesta separar la intención pura (funcional) de los detalles de implementación (técnico). SDD requiere una disciplina estricta.

Síntesis: El Desarrollador como Arquitecto de Sistemas

- **La Transformación:** Escribir Markdown estructurado y detallado hoy es el equivalente a compilar código ayer.

- **El Nuevo Rol:** El desarrollador ya no es un trabajador de línea de ensamblaje escribiendo sintaxis. Es un editor de sistemas, gestor de restricciones y validador de ejecución.

- **El Resultado:** Cuando la especificación dirige el desarrollo, los cambios bruscos de negocio dejan de ser reescrituras manuales traumáticas para convertirse en regeneraciones sistemáticas.